

Atividade 09 - Expressões com Variáveis

Andrei Formiga

March 02, 2026

Sumário

1. Introdução	3
2. A Linguagem EV	4
3. Alterações léxicas	5
4. Alterações na sintaxe	6
5. Análise Semântica e Verificação de Variáveis	7
5.1. Tabela de símbolos	7
6. Geração de código	9
7. Variações sintáticas	12
8. Artefato para entrega	13

1. Introdução

Agora que temos um compilador completo para expressões constantes, vamos adicionar a possibilidade de criar e usar variáveis nas expressões. Para isso, vai ser necessário mudar todos os estágios do compilador: análise léxica, análise sintática e geração de código.

A existência de variáveis na linguagem também cria a necessidade de gerenciar os nomes usados pelo programa, o que será usado para verificar se uma variável que aparece em uma expressão foi declarada anteriormente ou não. Essa verificação é feita na árvore sintática, e portanto após a análise sintática, configurando uma nova etapa do compilador: a *análise semântica* ou *análise contextual*.

2. A Linguagem EV

A linguagem EV (Expressões com Variáveis) adiciona a possibilidade de declarar e usar variáveis. O programa agora é uma série de zero ou mais declarações de variáveis, seguidas de uma expressão final, cujo resultado é o resultado do programa inteiro.

Por exemplo, o seguinte programa calcula o perímetro de um retângulo de largura 30 e comprimento 40:

```
l = 30;  
c = 40;  
= l + l + c + c
```

Um bom uso de variáveis pode tornar os programas mais fáceis de escrever e também mais fáceis de entender.

A gramática da linguagem EV é:

```
<programa> ::= <decl>* <result>  
<decl>      ::= <ident> '=' <exp> ';'   
<ident>      ::= <letra><letra_digito>*   
<result>     ::= '=' <exp>   
<exp>        ::= <exp_m> (('+' | '-' ) <exp_m>)*   
<exp_m>      ::= <prim> (('*' | '/' ) <prim>)*   
<prim>       ::= <num> | <ident> | '(' <exp> ')'  
<num>        ::= <digito><digito>*
```

(Nesta gramática, `<letra>` representa uma letra maiúscula ou minúscula, `<digito>` representa um dígito de 0 a 9, `<letra_digito>` representa uma letra maiúscula ou minúscula, ou um dígito).

Cada declaração começa com um identificador (usado como nome de variável) seguido por um sinal de igual e uma expressão. O valor da expressão deve ser calculado e atribuído à variável no lado esquerdo. A declaração termina com um ponto-e-vírgula. Por exemplo:

```
x = (7 + 4) * 12;
```

Expressões podem usar as variáveis que foram declaradas anteriormente. Como consequência, a expressão final pode usar todas as variáveis declaradas. Uma expressão que usa uma variável ainda não declarada deve causar um erro.

Um exemplo de programa completo:

```
x = (7 + 4) * 12;  
y = x * 3 + 11;  
= (x * y) + (x * 11) + (y * 13)
```

A variável `y` pode usar `x` na sua declaração, mas `x`, por ser a primeira declaração, não poderia usar nenhuma variável. A expressão final deve começar com um sinal de igual, e pode usar `x` e `y`. O resultado do programa é o resultado da expressão final, que é 60467.

Para implementar essa linguagem a partir do compilador para expressões constantes, é preciso alterar o analisador léxico, o analisador sintático e o gerador de código. Além disso, será necessário ter uma tabela de símbolos.

3. Alterações léxicas

Na linguagem EV existem três novos tipos de *token*: ponto-e-vírgula, sinal de igual (usado para atribuição) e identificadores. Um identificador é uma sequência de caracteres começando com uma letra (maiúscula ou minúscula), seguida de zero ou mais letras ou dígitos. O identificador é terminado por algum caractere que não seja letra ou dígito.

Note que uma sequência de letras e dígitos começando por um dígito (por exemplo, `237axy`) ainda é um erro léxico na linguagem EV.

4. Alterações na sintaxe

Podemos reaproveitar a parte de expressões da linguagem EC2, com uma alteração: um novo tipo de expressão primária é uma variável. A gramática da linguagem EV adiciona novos não-terminais que usam expressões como componentes.

Um programa é uma sequência de zero ou mais declarações, terminando por uma expressão final obrigatória. Cada declaração começa por um nome de variável, enquanto que a expressão final começa com um sinal de igual.

O analisador sintático deve começar pelo não-terminal `<programa>`. A função para analisar `<programa>` deve olhar o próximo *token* e, enquanto for um identificador (nome de variável), uma declaração deve ser reconhecida e guardada em uma coleção. Quando o próximo *token* for um sinal de igual, a análise deve reconhecer a expressão final. O pseudo-código para a função de análise de `<programa>` é:

```
programa():
    tok = olharProximoToken()
    declaracoes = []
    while tok.tipo == Identificador:
        dec = decl()
        declaracoes.adicionar(dec)
        tok = olharProximoToken()

    if tok != '=':
        erro

    expFinal = exp()

    return Programa(declaracoes, expFinal)
```

A função termina construindo o nó raiz da árvore de sintaxe abstrata, um nó `Programa` que deve guardar as declarações e a expressão final.

A função para reconhecer uma declaração também é simples: reconhecer um nome de variável, depois um sinal de igual, depois uma expressão, e por fim o ponto-e-vírgula. O nó da árvore sintática para uma declaração deve guardar o nome da variável e a expressão.

Nas funções para reconhecer expressões, a única diferença é que a função para analisar `<prim>` deve reconhecer a possibilidade de ser um nome de variável.

5. Análise Semântica e Verificação de Variáveis

Com as alterações discutidas até agora, o compilador vai poder fazer a análise sintática de qualquer programa na linguagem EV. Mas agora que a linguagem inclui variáveis, existem programas sintaticamente corretos na linguagem que apresentam um outro tipo de erros.

Na linguagem EV, o novo tipo de erro que pode acontecer é o uso de uma variável que não foi declarada.

Por exemplo, o programa a seguir está sintaticamente correto, mas não pode ser compilado pois tem duas referências a variáveis não declaradas:

```
x = 7 + y;  
y = x * 11;  
= x * y + z
```

A declaração de `x` tem uma referência à variável `y` antes de sua declaração; e a expressão final tem uma referência à variável `z` que não foi declarada.

(Em algumas linguagens, a declaração da variável `x` seria permitida, já que usa uma variável que está declarada no programa, embora depois da declaração de `x`. A referência a `z` na expressão de resultado continuaria sendo um erro, já que `z` não é declarada em nenhuma parte do programa.)

Para fazer a verificação das variáveis, é preciso percorrer a árvore sintática produzida para o programa e analisar cada declaração na ordem que elas aparecem no arquivo fonte da entrada. Cada declaração define uma variável com o valor de uma expressão. Para verificar a declaração, é preciso verificar se as variáveis usadas pela expressão foram declaradas antes; se não houver um erro de variável não-declarada, a variável declarada na expressão deve ser adicionada ao conjunto de variáveis declaradas.

Após fazer isso para cada declaração, na ordem que as declarações aparecem, resta apenas verificar se a expressão de resultado contém variáveis não-declaradas. Tanto nas declarações quanto na expressão de resultado, se for encontrada uma referência a alguma variável que não foi declarada antes, o compilador deve reportar um erro e parar o processo de compilação.

Como mencionado, uma referência a uma variável não declarada não é um erro sintático, pois a gramática não faz diferença entre variáveis declaradas ou não declaradas. A verificação de variáveis é parte de uma etapa de análise conhecida como *análise semântica* ou *análise contextual*.

Outro tipo de verificação comum na análise semântica é a verificação de tipos, que não é necessária na linguagem EV pois todas as variáveis são do mesmo tipo.

Uma questão aqui é: como verificar se uma variável já foi declarada? Para isso, vamos utilizar uma estrutura de dados chamada de *tabela de símbolos*.

5.1. Tabela de símbolos

Um compilador (ou interpretador) precisa lidar com muitos *símbolos*, que são os nomes de entidades do programa: variáveis, funções, classes, módulos, etc. Esses símbolos aparecem

nas várias etapas do compilador, e geralmente é necessário associar informações a cada um deles, para que o compilador possa saber como processar cada símbolo.

A estrutura de dados que guarda as informações de cada símbolo é normalmente chamada de *tabela de símbolos*, e é crucial para compiladores e interpretadores.

À medida que nosso compilador for incorporando mais características de linguagens de programação, e ficando mais complexo, usaremos a tabela de símbolos para guardar mais informações. No momento, para a linguagem EV, só precisamos guardar se a variável foi declarada ou não.

A tabela de símbolos é quase sempre implementada usando alguma estrutura de dados associativa, geralmente uma *hash table* ou outro tipo de mapeamento. Para guardar apenas se a variável foi declarada ou não, basta associar um valor booleano a cada nome (ou usar um *hash set* incluindo apenas as variáveis declaradas).

Vamos considerar o seguinte programa de exemplo:

```
x = 7 * 8;  
y = x * 11 - 17;  
= x * y + 33
```

Ao processar uma declaração como `x = 7 * 8`, já que a expressão que define o valor de `x` não tem nenhum erro, podemos concluir que, a partir desse ponto, a variável `x` vai estar declarada; logo, podemos incluir o identificador `x` na tabela de símbolos. Em seguida, na declaração de `y` existe uma referência a `x`, mas como essa variável já foi declarada, a expressão que define `y` também não possui erros. Então podemos incluir `y` na tabela de símbolos. Por fim, a expressão final possui referências a `x` e `y`, mas ambas estão na tabela de símbolos e portanto foram declaradas. O programa está correto, e podemos prosseguir para a geração do código.

A tabela de símbolos muitas vezes é uma variável global no programa, ou algo similar como um objeto *Singleton*. No caso da linguagem EV, a tabela de símbolos será usada apenas na análise semântica, mas em linguagens mais complexas a tabela pode também ser usada na geração de código, o que torna mais interessante ter uma tabela de símbolos acessível globalmente.

6. Geração de código

Se o programa está correto sintaticamente e passa a análise semântica (ou seja, não tem referências a variáveis que não foram declaradas), é possível para o compilador gerar código de destino para o programa.

No nível mais baixo, uma variável pode ser vista apenas como uma região da memória que é usada para armazenar o valor da variável. Para cada variável, é necessário reservar um espaço na memória do processo. Para ler ou alterar o valor dessa variável, basta utilizar o endereço reservado para ela.

Isso pode ser feito de forma simples pois o gerador de código pode utilizar o suporte a símbolos que existe no *assembler*. É possível usar diretivas do *assembler* para criar variáveis em um dos segmentos de dados.

Como mencionado em atividades anteriores, um arquivo objeto (o que inclui os executáveis) é dividido em seções, e todo arquivo precisa ter pelo menos três seções obrigatórias: TEXT, DATA e BSS. TEXT é a seção para código, DATA e BSS são seções para dados, com DATA sendo usada para dados inicializados e BSS para dados não inicializados.

Vamos usar a seção BSS para as variáveis no código gerado já que as variáveis não começam com valor no início da execução do programa; esses valores são calculados e atribuídos durante a execução.

Para declarar uma seção no GNU Assembler usamos a diretiva `.section`. Já usamos no modelo de código gerado a definição da seção TEXT:

```
.section .text
```

Os compiladores até agora usam apenas a seção TEXT (embora o código de suporte no arquivo `runtime.s` utilize uma região de memória como *buffer* de impressão).

Agora vai ser necessário declarar uma seção BSS e declarar as variáveis dentro dessa seção, de preferência antes da seção TEXT. Cada variável (um inteiro de 64 bits) pode ser declarada com uma diretiva `.lcomm` usando o nome da variável e o tamanho da variável em bytes. Por exemplo, se formos declarar duas variáveis de nomes `x` e `y`, as diretivas devem ser:

```
.section .bss
.lcomm x, 8
.lcomm y, 8
```

Para usar a variável, basta usar o nome em instruções MOV e outras instruções que permitem operandos em memória. O código para uma declaração vai ser o código para a expressão do lado direito, e ao final uma instrução para passar o resultado que está em RAX para a variável. Por exemplo, para atribuir o valor do registrador RAX para a variável `x` declarada na seção BSS, a instrução é:

```
mov %rax, x
```

Para traduzir uma referência a uma variável, basta usar um MOV da variável para RAX (que é o local onde o resultado das expressões é colocado). Se uma expressão usa a variável `x`, o código gerado para essa referência é:

```
mov x, %rax
```

O processo de geração de código é:

1. Varrer a parte de declarações na árvore sintática para determinar quais variáveis precisam ser declaradas como símbolos no arquivo *assembly*. Para cada variável, gerar uma diretiva `.lcomm` que será incluída na seção BSS.
2. Varrer a parte de declarações novamente, gerando o código para a expressão de cada declaração; ao final do código gerado para a expressão, adicionar um `+mov+` para copiar o resultado da expressão para a variável. O código de cada expressão deve ser colocado em sequência após o rótulo `._start`.
3. Gerar código para a expressão de resultado; esse código deve ser adicionado logo após o código para as expressões de declarações.

Assim, o modelo do arquivo *assembly* agora tem duas partes que serão preenchidas pelo compilador: a parte para declaração das variáveis usando diretivas `.lcomm`, e a parte para o código gerado para as expressões:

```
# Modelo para arquivo de saida

.section .bss
# diretivas para as variaveis

.section .text
.globl _start

_start:
# codigo gerado para as expressoes

call imprime_num
call sair

.include "runtime.s"
```

Para ver um exemplo completo, o seguinte programa é a tradução do pequeno programa que está nos comentários:

```
# Programa-fonte:
# x = 11 * 7;
# y = x * 3 + 5;
# z = x * y

.section .bss
.lcomm x, 8
.lcomm y, 8

.section .text
.globl _start

_start:
# x = 11 * 7;
mov $11, %rax
push %rax
mov $7, %rax
```

```
pop %rbx
mul %rbx
mov %rax, x
```

```
# y = x * 3 + 5;
mov x, %rax
push %rax
mov $3, %rax
pop %rbx
mul %rbx
push %rax
mov $5, %rax
pop %rbx
add %rbx, %rax
mov %rax, y
```

```
# = x * y
mov x, %rax
push %rax
mov y, %rax
pop %rbx
mul %rbx
```

```
call imprime_num
call sair
```

```
.include "runtime.s"
```

7. Variações sintáticas

Nas atividades anteriores a linguagem fonte do compilador era uma linguagem de expressões, e a sintaxe para expressões aritméticas geralmente não muda entre diferentes linguagens. Agora que a linguagem tem declarações de variáveis e um resultado final, é possível usar gramáticas diferentes para a linguagem de entrada do compilador, de acordo com o interesse do grupo.

A sintaxe proposta neste guia foi criada para simplificar a análise léxica e a análise sintática. Com um pouco mais de trabalho, é possível usar uma sintaxe menos concisa, mas talvez mais legível.

Vamos considerar o seguinte programa de exemplo:

```
x = (7 + 4) * 12;  
y = x * 3 + 11;  
= (x * y) + (x * 11) + (y * 13)
```

Uma mudança simples é alterar o sinal de igual na expressão final para uma palavra-chave:

```
x = (7 + 4) * 12;  
y = x * 3 + 11;  
return (x * y) + (x * 11) + (y * 13)
```

Quando a linguagem tem palavras-chave (*keywords*), é comum que o analisador léxico trate as palavras-chave de forma diferente dos identificadores, e que exista um tipo de *token* separado para palavras-chave. Isso simplifica a análise sintática, e é recomendado no caso de variantes da sintaxe que usam palavras-chave.

Outra possibilidade é usar uma sintaxe inspirada pela linguagem Pascal:

```
var x = (7 + 4) * 12;  
var y = x * 3 + 11;  
begin  
    (x * y) + (x * 11) + (y * 13)  
end
```

Ou uma sintaxe inspirada pela linguagem C:

```
int x = (7 + 4) * 12;  
int y = x * 3 + 11;  
main() {  
    (x * y) + (x * 11) + (y * 13);  
}
```

Outras variações podem ser utilizadas. Note que todas essas variações sintáticas não alteram os componentes de cada parte do programa, o que significa que as árvores de sintaxe abstrata serão as mesmas em todos os casos. Isso reforça a ideia que vimos que a árvore de sintaxe abstrata realmente abstrai os detalhes da sintaxe *concreta* (que é a gramática reconhecida pelo analisador sintático).

8. Artefato para entrega

Cada grupo deve entregar o compilador para a linguagem EV ou alguma variante sintática indicada. Caso o grupo tenha decidido implementar uma variação sintática da linguagem EV, o projeto deve incluir na documentação qual foi a sintaxe utilizada, de preferência com a especificação da gramática completa.

O projeto deve incluir testes que usam variáveis das formas permitidas, e também testes que demonstram que o compilador consegue detectar e reportar o uso de variáveis que não foram declaradas.

O projeto também deve incluir documentação de como usar o compilador e como executar os testes.